Files and File Systems

What are Files?

- Files are data collections created by users.
- The File System is a crucial component of the OS for users.

Desirable Properties of Files

- **Long-term existence:** Files are stored on disk or other secondary storage and persist even when a user logs off.
- **Sharable between processes:** Files have names and access permissions for controlled sharing.
- **Structure:** Files can have an internal structure, organized hierarchically or in more complex ways, depending on the file system.

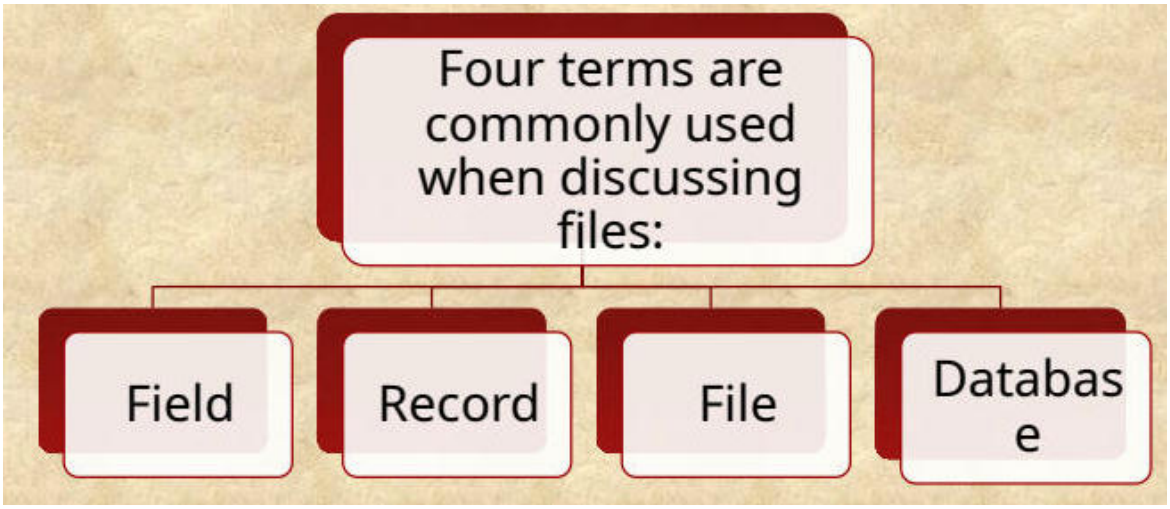
Operations on Files

File systems provide a means to store data organized as files and offer functions to operate on them. Typical operations include:

- Open
- Close
- Read
- Write
- Create
- Delete

They also maintain a set of attributes associated with each file.

File Structure



The image above illustrates how files can be structured either as a collection of records or as a sequence of bytes. UNIX, Linux, Windows, and macOS treat files as a sequence of bytes. Other OSs, like those in IBM mainframes, use a collection-of-records approach, which is useful for databases.

Structure Terms

Field

Basic element of data that contains a single value. Examples include an employee's last name or date of birth. It is characterized by its length and data type (e.g., ASCII string, decimal) and can be of fixed or variable length.

Record

Collection of related fields treated as a unit by an application program. One field is the key, which is a unique identifier. An employee record might contain fields such as name, social security number, job classification, and date of hire. Records can be of fixed or variable length depending on the design.

File

Collection of similar records treated as a single entity by the user and applications. Files are referenced by name, and access control restrictions usually apply at the file level.

Database

Consists of one or more types of files. It is a collection of related data with explicit relationships among the elements, designed for use by multiple applications.

File Management System Objectives

A file management system is system software providing services to users and applications for file usage. Users and applications typically access files only through the file management system.

Key objectives include:

- Meeting user data management needs.
- Ensuring data validity.
- Optimizing performance.
- Providing I/O support for various storage device types.
- Minimizing data loss or destruction.
- Offering a standardized set of I/O interface routines to user processes.
- Supporting I/O for multiple users in multi-user systems.

File System Architecture

The architecture of a file system is organized into layers, each with specific responsibilities. From the highest to the lowest level, these layers are:

- **File Formats**: Access methods provide the interface to users.
- **Logical I/O**: Enables users and applications to access records, providing general-purpose record I/O capability and maintaining basic data about files.
- **Basic I/O Supervisor**: Responsible for all file I/O initiation and termination. It manages control structures, device I/O scheduling, and file status, selecting devices for I/O and assigning I/O buffers and secondary memory.
- **Basic File System**: Also known as the physical I/O level, it is the primary interface with the external environment, handling the placement of blocks on secondary storage and buffering blocks in main memory.
- **Device Drivers**: Communicate directly with peripheral devices, initiating I/O operations and processing I/O requests.

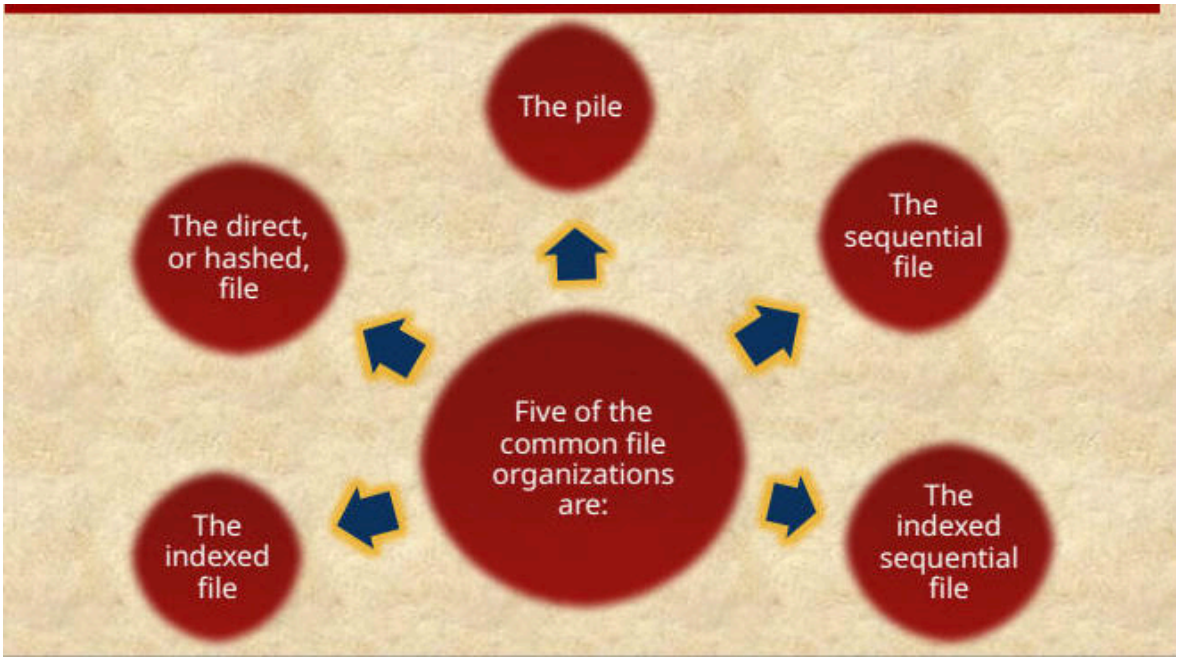
File Organization and Access

File organization refers to the logical structuring of records based on how they are accessed. Important criteria for choosing a file organization include:

- Short access time
- Ease of update
- Economy of storage
- Simple maintenance
- Reliability

The relative priority of these criteria depends on the application.

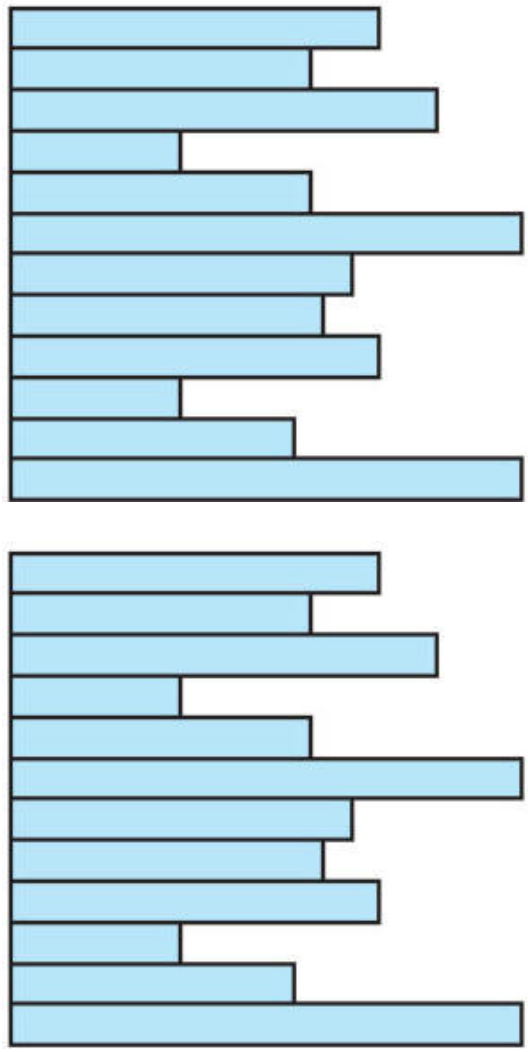
File Organization Types



The image above presents common file organizations that include: the pile, the sequential file, the indexed sequential file, the indexed file, and the direct or hashed file.

The Pile

- The least complicated form of file organization.
- Data is collected in the order it arrives to accumulate the mass of data and save it.
- Records may have different fields or similar fields in different orders.
- Each record consists of one burst of data.

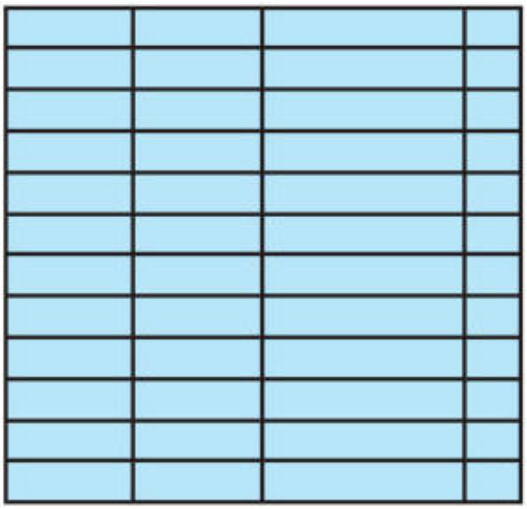


Horizontal bar chart with nine light blue bars outlined in black.

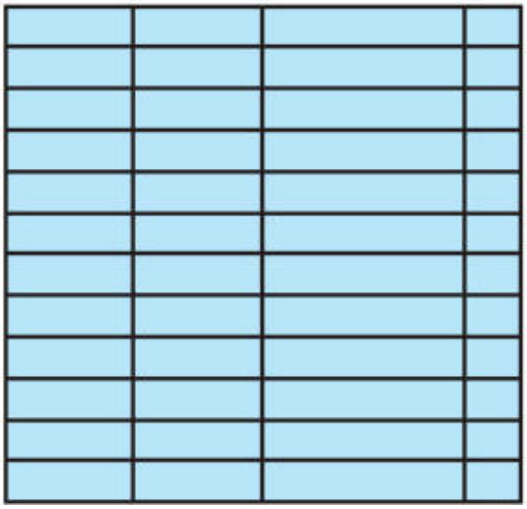
- Record access requires an exhaustive search because there is no structure to the pile file. To find a record with a specific field and value, each record must be examined until the desired one is found or the entire file has been searched.

The Sequential File

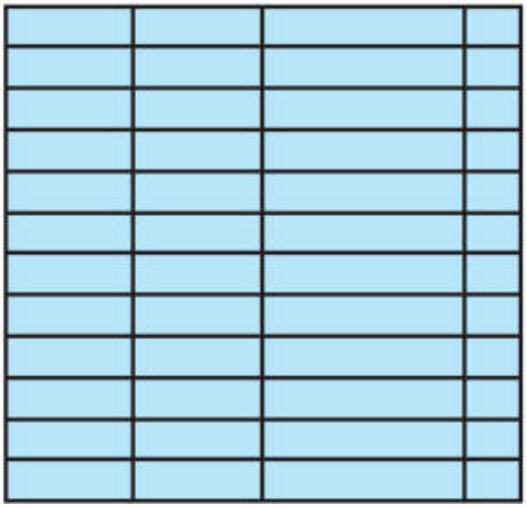
- The most common form of file structure.
- Records consist of the same number of fixed-length fields in a particular order.



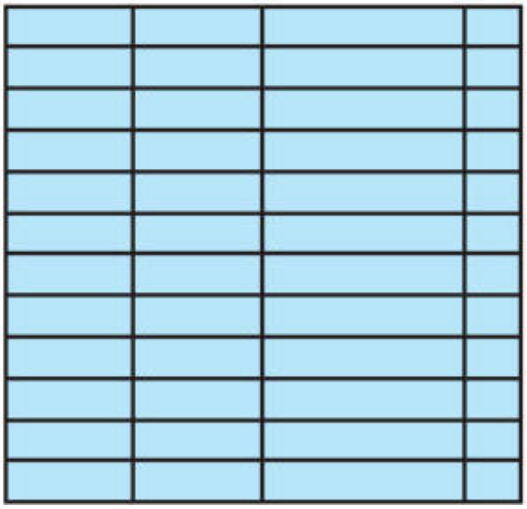
- A fixed format is used for records where all records are of the same length.



- One field, usually the first, is the **key field**, which uniquely identifies the record. Key values for different records are always different.



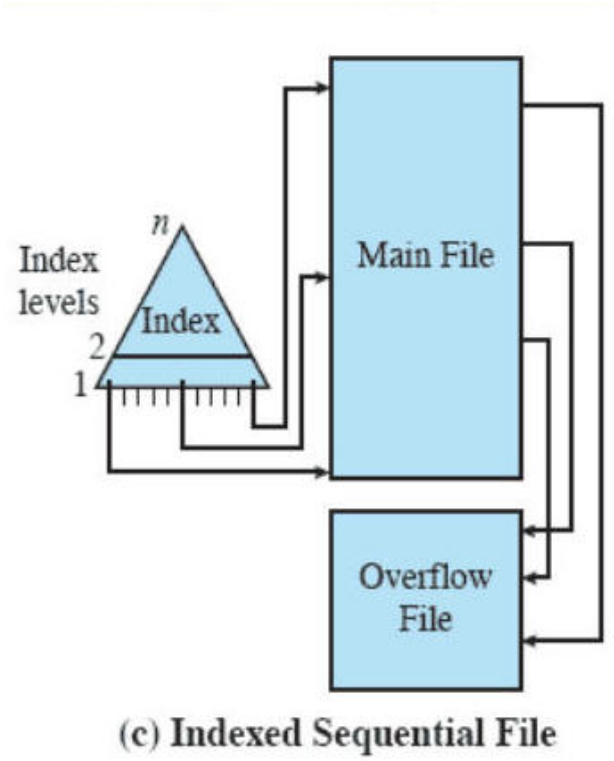
- Records are stored in **key sequence**, which means alphabetical order for a text key and numerical order for a numerical key.



- Typically used in batch applications involving the processing of all records (e.g., billing or payroll).
- Easily stored on tape as well as disk.

Indexed-Sequential File

- Maintains the key characteristic of the sequential file; records are organized in sequence based on a key field.
- Two features are added:
    1. An index to support random access.
    2. An overflow file.

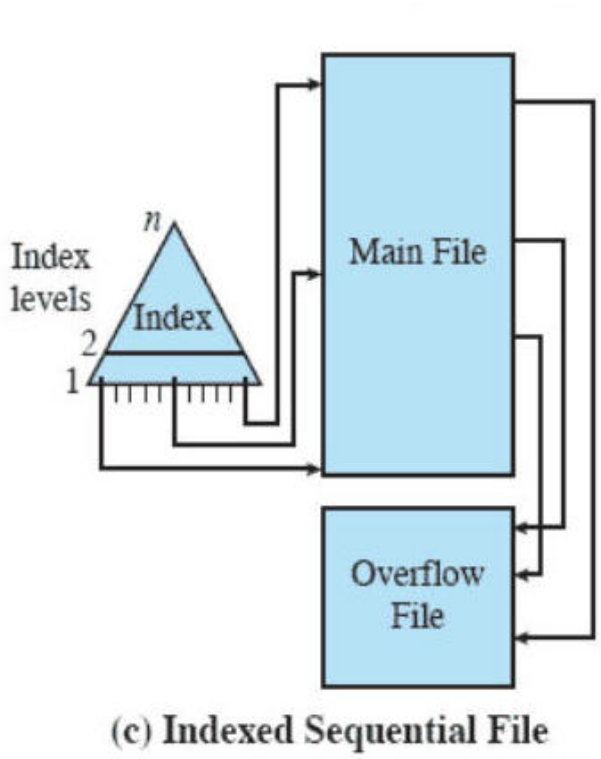


(c) Indexed Sequential File

Each record in the index file consists of two fields:

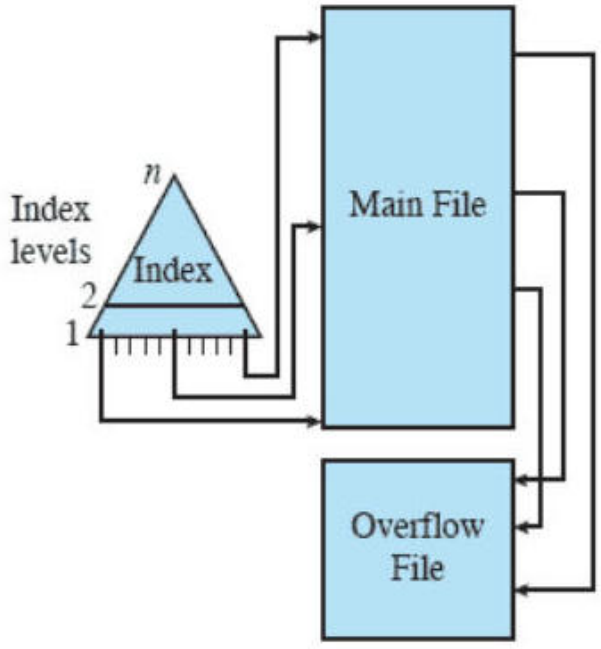
1. A key field, which is the same as the key field in the main file.
2. A pointer into the main file.

To find a specific field, the index is searched to find the highest key value that is equal to or precedes the desired key value. The search continues in the main file at the location indicated by the pointer. This reduces the average search length.



(c) Indexed Sequential File

Consider a sequential file with 1 million records. Searching for a particular key value will require, on average, one half million record accesses. If an index containing 1,000 entries is constructed, with the keys in the index more or less evenly distributed over the main file, the search length is reduced.



(c) Indexed Sequential File

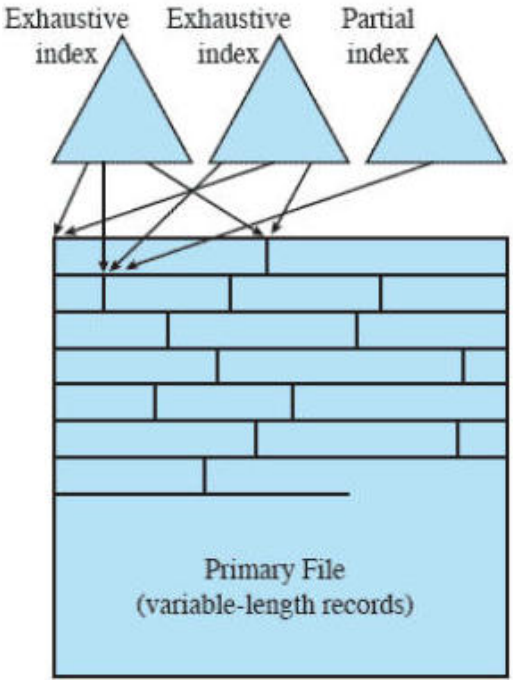
Each record in the main file contains an additional field (not visible to the application) that is a pointer to the overflow file.

When a new record is to be inserted into the file:

1. It is added to the overflow file.
2. The record in the main file that immediately precedes the new record in logical sequence is updated to contain a pointer to the new record in the overflow file.

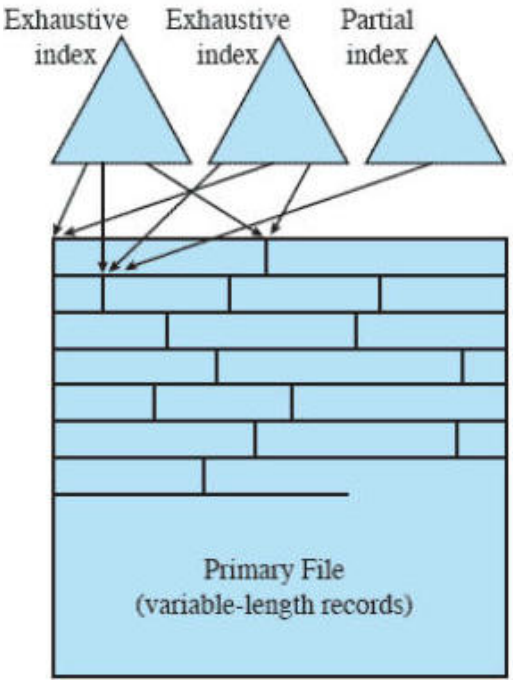
Indexed File

- Records are accessed only through their indexes.
- A structure is needed that employs multiple indexes, one for each type of field that may be the subject of a search.
- There is no restriction on the placement of records as long as a pointer in at least one index refers to that record.
- Variable-length records can be employed.



Two types of indexes are used:

- An **exhaustive index** contains one entry for every record in the main file.
- A **partial index** contains entries to records where the field of interest exists (useful with variable-length records, where some records may not contain all fields).



Indexed files are used mostly in applications where timeliness is critical and where data are rarely processed exhaustively, such as airline reservation systems and inventory control systems. When a new record is added to the main file, all of the index files must be updated.

Direct or Hashed File

- The direct file makes use of **hashing** on the key value.
- There is no concept of sequential ordering.

Examples include:

- Directories
- Pricing tables
- Schedules
- Name lists

Often used where:

- Very rapid access is required.
- Fixed-length records are used.
- Records are always accessed one at a time.

File Directory Information

Associated with any file management system and collection of files is a **file directory**.

- The directory contains information about the files, including attributes, location, and ownership.
- The directory is itself a file, accessible by various file management routines.

The information typically stored in the directory for each file in the system includes its name, type, address, current length, maximum length, date last accessed, date last updated, owner ID, and access control information.

Operations Performed on a Directory

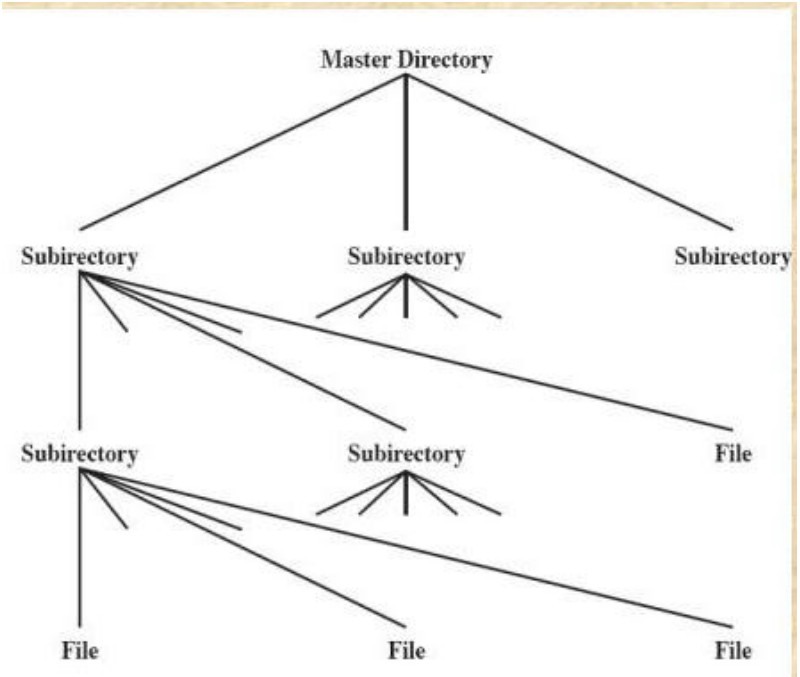
To understand the requirements for a file structure, consider the types of operations that may be performed on the directory:

Directory Structures

There are a few different directory structures:

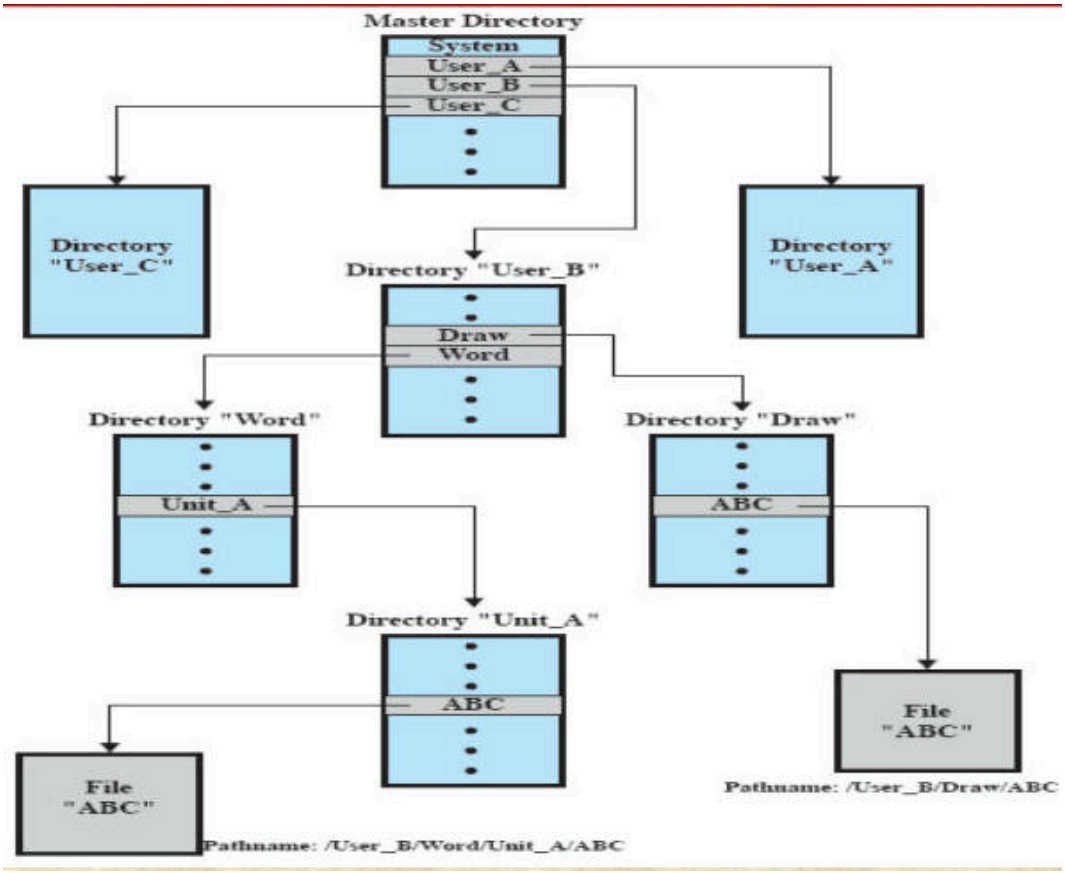
Two-Level Scheme

- Master directory has an entry for each user directory providing address and access control information
- There is one directory for each user and a master directory
- Names must be unique only within the collection of files of a single user
- Each user directory is a simple list of the files of that user
- File system can easily enforce access restriction on directories

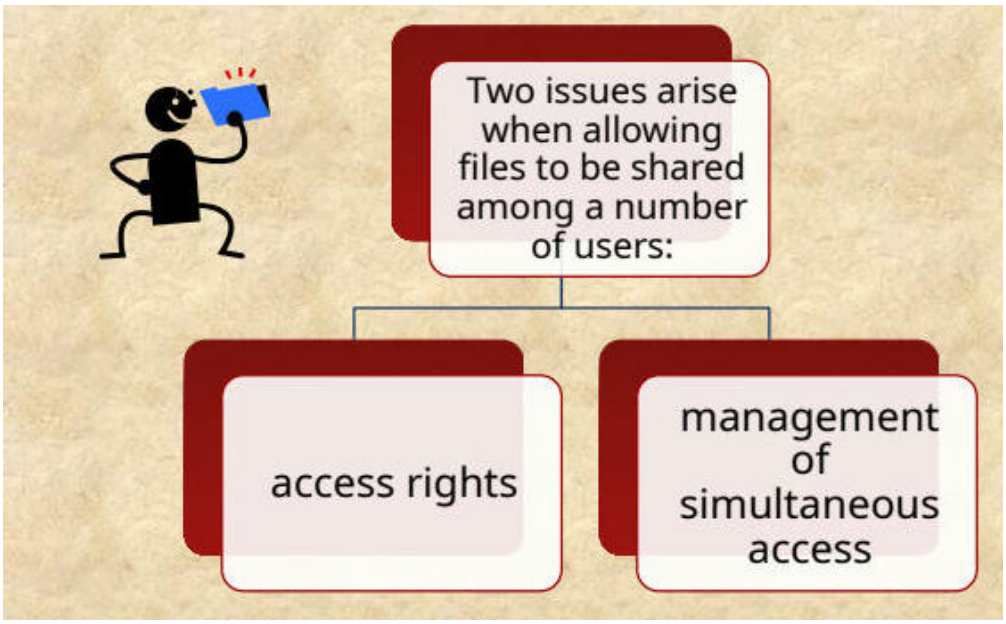


Tree-Structured Directory

- Master directory with user directories underneath it
- Each user directory may have subdirectories and files as entries



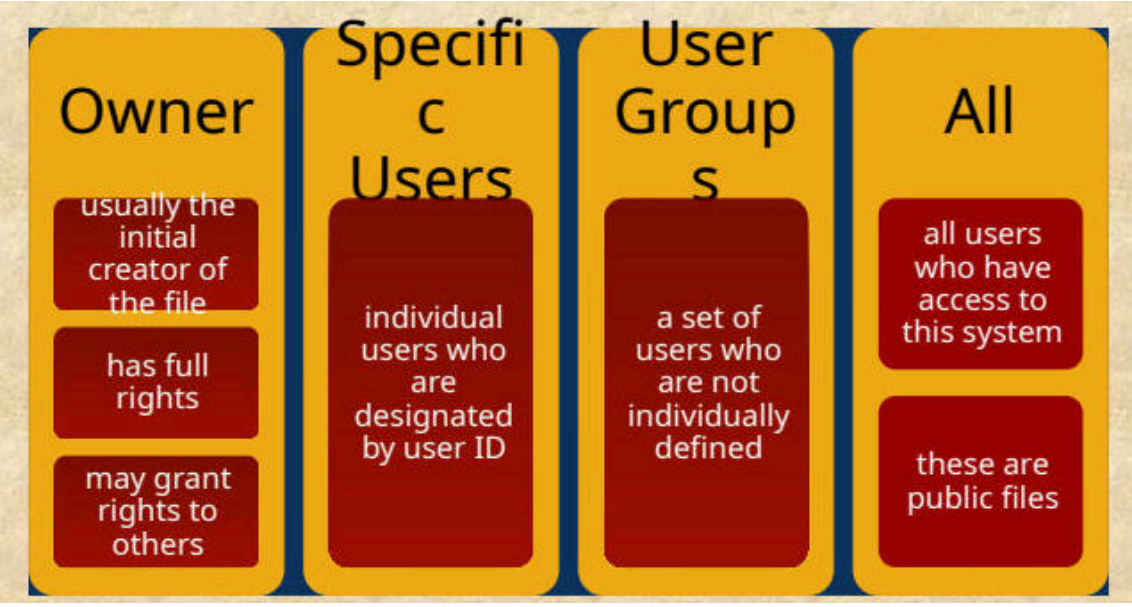
File Sharing



Access Rights

- None: The user cannot read the user directory that includes the file.
- Knowledge: The user can determine that the file exists and who its owner is and can then petition the owner for additional access rights.
- Execution: The user can load and execute a program but cannot copy it.
- Reading: The user can read the file for any purpose.
- Appending: The user can add data to the file but cannot modify or delete any of the file's contents.
- Updating: The user can modify, delete, and add to the file's data.
- Changing Protection: The user can change the access rights granted to other users.
- Deletion: The user can delete the file from the file system.

User Access Rights



Secondary Storage Management

- On secondary storage, a file consists of a collection of blocks.
- The operating system or file management system is responsible for allocating blocks to files.

This raises two management issues:

1. Space on secondary storage must be allocated to files.
2. It is necessary to keep track of the space available for allocation.

File Allocation

- Disks are divided into physical blocks (sectors on a track).
- Files are divided into logical blocks (subdivisions of the file).
- Logical block size = some multiple of a physical block size.

The operating system or file management system is responsible for allocating blocks to files. Space is allocated to a file as one or more **portions** (contiguous set of allocated disk blocks). A portion is the logical block size. A **file allocation table (FAT)** is a data structure used to keep track of the portions assigned to a file.

Preallocation vs. Dynamic Allocation

- A **preallocation** policy requires that the maximum size of a file be declared at the time of the file creation request.
    - For many applications, it is difficult to estimate reliably the maximum potential size of the file.
    - Tends to be wasteful because users and application programmers tend to overestimate size.
- **Dynamic allocation** allocates space to a file in portions as needed.

Portion Size

In choosing a portion size, there is a trade-off between efficiency from the point of view of a single file versus overall system efficiency.

Items to be considered:

1. Contiguity of space increases performance, especially for Retrieve_Next operations, and greatly for transactions running in a transaction-oriented operating system.
2. Having a large number of small portions increases the size of tables needed to manage the allocation information.
3. Having fixed-size portions simplifies the reallocation of space.

Summarizing the Alternatives

Two major alternatives:

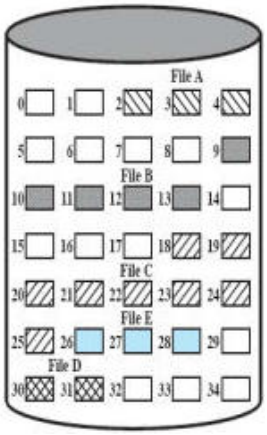
- Variable, large contiguous portions:
  - Provides better performance.
  - The variable size avoids waste.
  - The file allocation tables are small.
- Blocks:
  - Small fixed portions provide greater flexibility.
  - They may require large tables or complex structures for their allocation.
  - Contiguity has been abandoned as a primary goal.
  - Blocks are allocated as needed.

File Allocation Methods

Method	Description
Contiguous Allocation	A single contiguous set of blocks is allocated to a file at the time of file creation. This is a preallocation strategy using variable-size portions. The file allocation table needs just a single entry for each file, showing the starting block and the length of the file.
Chained Allocation	Allocation is on an individual block basis, with each block containing a pointer to the next block in the chain. The file allocation table needs just a single entry for each file, showing the starting block and the length of the file. Each block contains a pointer to the next block. Allocation is on an individual block basis

Contiguous File Allocation

- Contiguous allocation is the best from the point of view of the individual sequential file.
- Multiple blocks can be read in at a time to improve I/O performance for sequential processing.
- It is also easy to retrieve a single block.
  - For example, if a file starts at block b, and the ith block of the file is wanted, its location on secondary storage is simply $b + i – 1$.



Problems with Contiguous Allocation

- External fragmentation will occur, making it difficult to find contiguous blocks of space of sufficient length.
- From time to time, it will be necessary to perform a compaction algorithm to free up additional space on the disk.

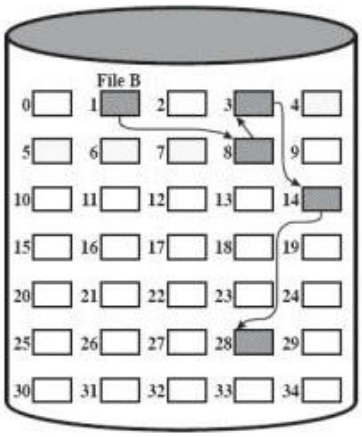
Chained Allocation

- Allocation is on an individual block basis.
- Each block contains a pointer to the next block in the chain.
- The file allocation table needs just a single entry for each file, showing the starting block and the length of the file.

Chained Allocation

Chained allocation allocates blocks as needed, simplifying the block selection process to choosing any free block and adding it to a chain.

The image below illustrates how File B is stored in non-contiguous blocks on the disk, demonstrating chained allocation.



Key aspects of chained allocation include:

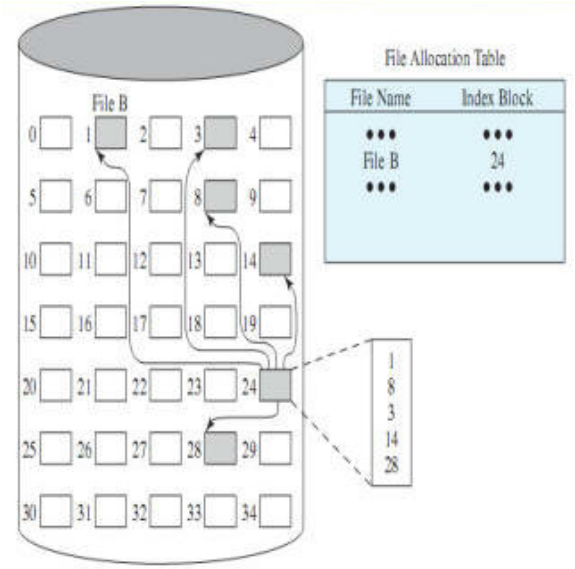
- No external fragmentation because only one block is needed at a time.
- Better suitability for sequential files.

A drawback is that chaining does not inherently consider the principle of locality. Some systems periodically consolidate files to address this.

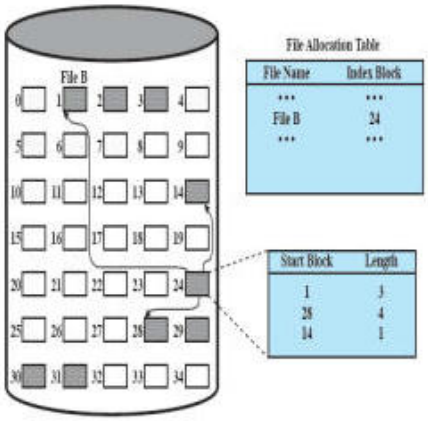
Indexed Allocation

In indexed allocation, the file allocation table contains a separate one-level index for each file. The index has one entry for each portion allocated to the file. The file index is kept in a separate block, and the entry for the file in the file allocation table points to that block.

The image below depicts a file system representation, specifically illustrating how files are allocated on a disk using a File Allocation Table (FAT).



The image below illustrates a file allocation table used in a computer's file system, showcasing the relationship between files, blocks, and index allocation.



To enhance efficiency, the index should be based on variable-size portions rather than blocks, providing efficient support for all file allocation methods.

Free Space Management

Just as allocated space must be managed, so must the unallocated space. To perform file allocation, it is necessary to know which blocks are available, meaning that a **disk allocation table** is needed in addition to a **file allocation table**.

Bit Tables (Bit Vectors)

This method employs a **vector** containing one **bit** for each **block** on the disk.

- A **0** corresponds to a **free block**.
- A **1** corresponds to a **block in use**.

For the disk layout of Figure 12.9, a vector of length 35 is needed and would have the following value:

```
00111000011111000011111111111011000
```

Advantages:

- Works well with any file allocation method.
- Relatively easy to find one or a contiguous group of free blocks.
- It is as small as possible.

Chained Free Portions

Free portions are linked together using a pointer and length value in each free portion.

Advantages:

- Negligible space overhead, eliminating the need for a disk allocation table and only requiring a pointer to the chain's start and the length of the first portion.
- Suited to all file allocation methods

Disadvantages:

- Leads to fragmentation.
- Every time you allocate a block, you need to read the block first to recover the pointer to the new first free block before writing data to that block.
- Treats free space as a file and uses an index table as it would for file allocation.

Indexing

For efficiency, the index should be on the basis of variable-size portions rather than blocks.

Free Block List

Each block is assigned a number sequentially, and the list of the numbers of all free blocks is maintained in a reserved portion of the disk. Depending on the size of the disk, either 24 or 32 bits will be needed to store a single block number the size of the free block list is 24 or 32 times the size of the corresponding bit table and must be stored on disk.

The free block list must be stored on disk rather than in main memory because it is too large.

There are two effective techniques for storing a small part of the list in main memory:

- The list can be treated as a **push-down stack**, with the first few thousand elements of the stack kept in main memory.
    - When a new block is allocated, it is popped from the top of the stack, which is in main memory.
    - Similarly, when a block is deallocated, it is pushed onto the stack.
    - There only has to be a transfer between disk and main memory when the in-memory portion of the stack becomes either full or empty.
- The list can be treated as a **FIFO queue**, with a few thousand entries from both the head and the tail of the queue in main memory.
    - A block is allocated by taking the first entry from the head of the queue.
    - A block is deallocated by adding it to the end of the tail of the queue.
    - There only has to be a transfer between disk and main memory when either the in-memory portion of the head of the queue becomes empty or the in-memory portion of the tail of the queue becomes full.

Review

- File systems can support files organized as a sequence of bytes or as a sequence of records.
- Access methods depend on file organization.
- Disk storage of files can be contiguous, linked, or indexed.
- Logical blocks of a file are mapped to one or more disk sectors to create physical blocks.
- File Allocation Tables map files to disk locations.

UNIX File Management

In the UNIX file system, six types of files are distinguished:

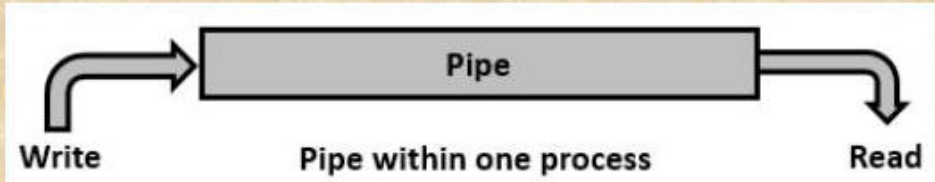
1. **Regular, or Ordinary**
    - Contain information entered in them by a user, an application program, or a system utility program.
    - An ordinary file is a file on the system that contains data, text, or program instructions.
    - Contains arbitrary data in zero or more data blocks.
    - The file system does not impose any internal structure to a regular file but treats it as a stream of bytes.
2. **Directory**
    - Contains a list of file names plus pointers to associated inodes (index nodes).
    - Directories are hierarchically organized.
3. **Special**
    - Contains no data but provides a mechanism to map physical devices to file names.
    - The file names are used to access peripheral devices, such as terminals and printers.
    - Each I/O device is associated with a special
4. **Named pipes**

    An interprocess communications facility. A pipe file buffers data received in its input so that a process that reads from the pipe's output receives the data on a first-in-firstout basis.

The image below depicts a horizontal rectangle with a textured, yellow-orange background that resembles parchment or aged paper.

- A pipe is a section of shared memory that processes use for communication. The process that creates a pipe is the pipe server, and a process that connects to a pipe is a pipe client. One process writes information to the pipe, then the other process reads the information from the pipe.

  The image below depicts a simple diagram of a pipe within one process, showcasing the flow of data.



5. Special

   - Used to represent a real physical device such as a printer, tape drive, or terminal used for Input/Output (I/O) operations.
   - Device or special files are used for device Input/Output(I/O) on UNIX and Linux systems.

6. Sockets

   - Sockets – A Unix socket (or Inter-process communication socket) is a special file that allows for advanced inter-process communication.
   - A Unix Socket is used in a client-server application framework.

7. Symbolic links

   A data file that contains the name of the file it is linked to.

   - A symbolic link is used for referencing some other file of the file system.
   - A symbolic link is also known as a Soft link and contains a text form of the path to the file it references.
   - To an end user, a symbolic link will appear to have its own name, but when you try reading or writing data to this file, it will instead reference these operations to the file it points to.
   - If we delete the soft link itself, the data file would still be there. If we delete the source file or move it to a different location, the symbolic file will not function properly.
   - In long-format output of ls –l, symbolic links are marked by the “l” symbol (that’s a lower case L).

To find out file types in Linux, check out https://www.geeksforgeeks.org/how-to-find-out-file-types-inlinux/.

Inodes

Modern UNIX operating systems support multiple file systems but map all of these into a uniform underlying system for supporting file systems and allocating disk space to files. All types of UNIX files are administered by the OS by means of inodes.

> The inode (index node) is a data structure in a Unix-style file system that describes a file-system object such as a file or a directory.

Each inode stores the attributes and disk block locations of the object's data. An inode (index node) is a control structure that contains the key information needed by the operating system for a particular file.

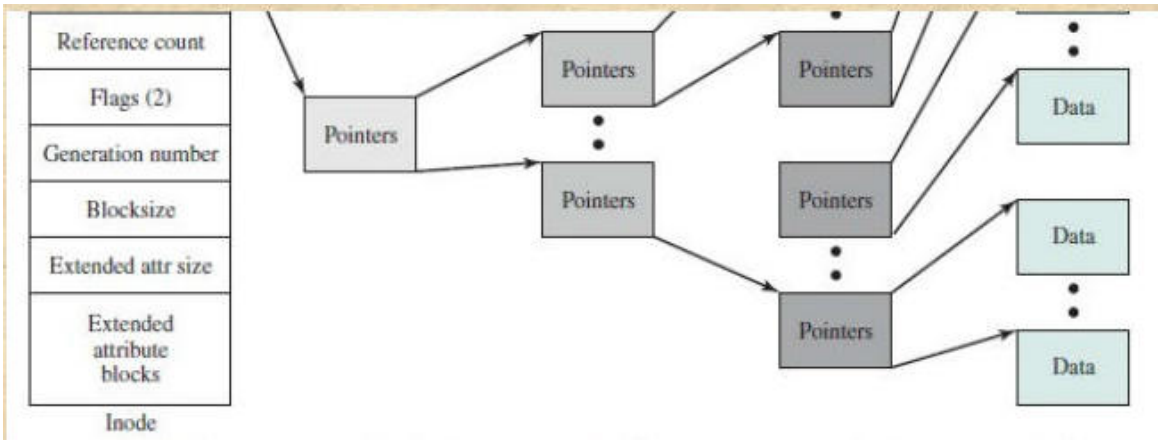
FreeBSD Inode and File Structure

- The exact inode structure varies from one UNIX implementation to another.

The FreeBSD inode structure includes the following data elements:

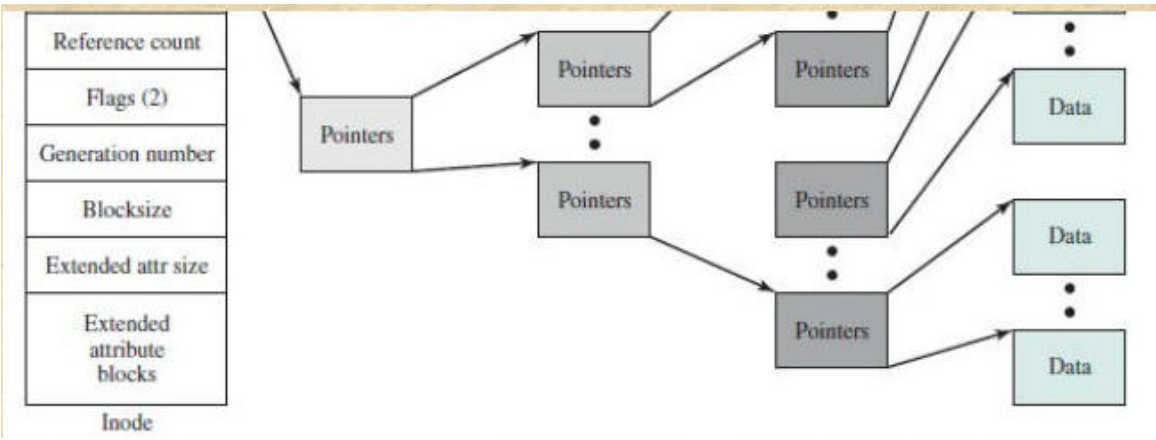
- The type and access mode of the file
- The file's owner and group-access identifiers
- The time that the file was created, when it was most recently read and written, and when its inode was most recently updated by the system
- The size of the file in bytes
- A sequence of block pointers
- The number of physical blocks used by the file, including blocks used to hold indirect pointers and attributes

  The image below depicts a detailed diagram of an inode data structure and its associated pointers, which are used to access data blocks on a file system.



- The number of directory entries that reference the file
- The kernel and user-settable flags that describe the characteristics of the file
- The generation number of the file (a randomly selected number assigned to the inode each time the latter is allocated to a new file)
- The blocksize of the data blocks referenced by the inode

  The image below depicts a detailed diagram of a file system structure, specifically illustrating the organization and relationships between various components.

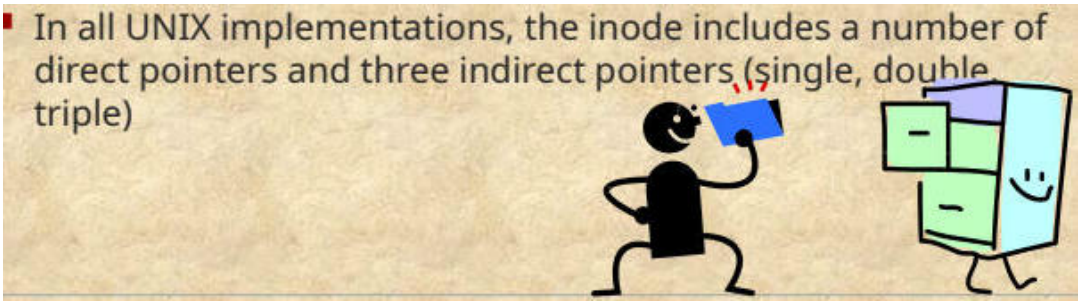


- On the disk, there is an inode table, or inode list, that contains the inodes of all the files in the file system.
- When a file is opened, its inode is brought into main memory and stored in a memory-resident inode table.

File Allocation-Unix

File allocation is done on a block basis. Allocation is dynamic, as needed, rather than using preallocation. An indexed method is used to keep track of each file, with part of the index stored in the inode for the file.

The image below depicts a cartoon-style illustration of two characters and accompanying text.

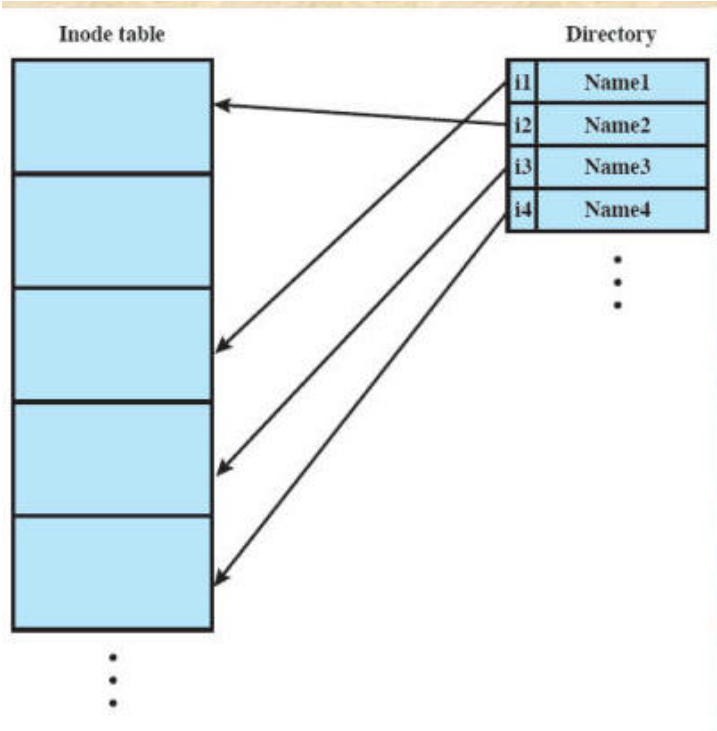


UNIX- Directories and Inodes

Directories are structured in a hierarchical tree. Each directory can contain files and/or other directories. A directory inside another directory is referred to as a subdirectory.

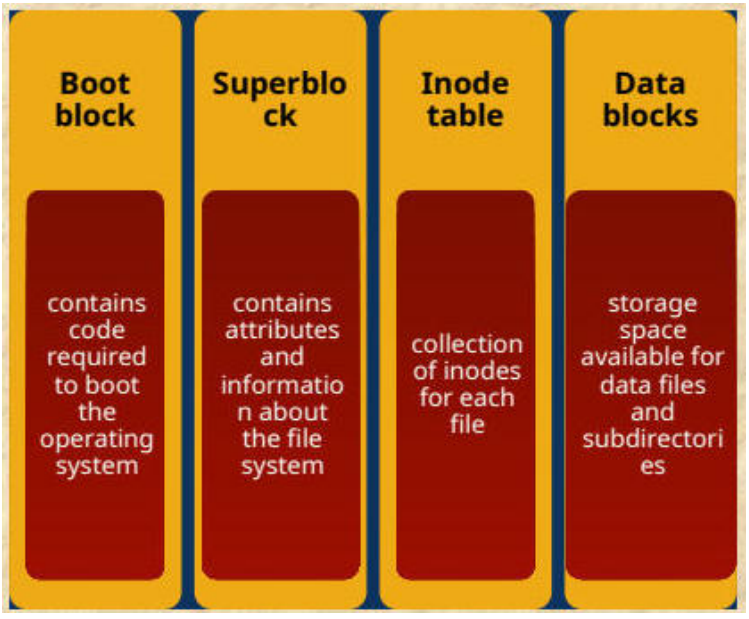
A directory is simply a file that contains a list of file names plus pointers to associated inodes. Each directory entry (dentry) contains a name for the associated file or subdirectory plus an integer called the i-number (index number). When the file or directory is accessed, its inumber is used as an index into the inode table.

The image below depicts a diagram illustrating the relationship between an inode table and a directory in a file system.



Volume Structure

The image below presents a detailed breakdown of the structure of a file system, divided into four distinct sections.



A UNIX file system resides on a single logical disk or disk partition and is laid out with the following elements:

- Boot block
- Superblock
- Inode table
- Data blocks

UNIX File Access Control

Access Control Lists in UNIX

FreeBSD allows the administrator to assign a list of UNIX user IDs and groups to a file. Any number of users and groups can be associated with a file, each with three protection bits (read, write, execute). A file may be protected solely by the traditional UNIX file access mechanism. FreeBSD files include an additional protection bit that indicates whether the file has an extended ACL.

Linux Virtual File System (VFS)

Linux includes a versatile and powerful file-handling facility designed to support a wide variety of file management systems and file structures and presents a single, uniform file system interface to user processes.

- Defines a common file model that is capable of representing any conceivable file system's general feature and behavior.
- VFS Assumes files are objects that share basic properties regardless of the target file system or the underlying processor hardware.

The Role of VFS Within the Kernel

1. A user process issues a file system call (e.g., read) using the VFS file scheme.
2. The VFS converts this into an internal (to the kernel) file system call that is passed to a mapping function for a specific file system [e.g., IBM's Journaling File System (JFS)].
3. The mapping function is simply a mapping of file system functional calls from one scheme to another.

Primary Object Types in VFS

VFS is an object-oriented scheme. Because it is written in C, rather than a language that supports object programming (such as C++ or Java), VFS objects are implemented simply as C data structures.

Object Type	Description
Superblock Object	Represents a specific mounted file system
Inode Object	Represents a specific file
Dentry Object	Represents a specific directory entry
File Object	Represents an open file associated with a process

window.MathJax = {

```
tex: {
  inlineMath: [['$', '$'], ['\\(', '\\)']],
  displayMath: [['$$', '$$'], ['\\[', '\\]']]
}
```

};